

Making a Rule-Heavy Game Evaluatable: Oracles, Value Networks and LLM Post-Training on Twilight Struggle

2026-10-08

Technical report, 2026-10-08. Code, data and the leaderboard: github.com/bingyang1132/ts-eval (built on github.com/bingyang1132/ts-env).

Abstract

Twilight Struggle is a two-player wargame with hidden hands, dice and about 360 atomic decisions in an average game. We turn it into two evaluation tasks for language models: 847 held-out single decisions scored by an oracle, and full games against a scripted greedy baseline. Two oracles were rejected: rollouts to game end had a test-retest correlation of 0.22, and a reliable one-turn heuristic ($r = 0.77$) trained policies that held scoring cards and lowered DEFCON to 1 themselves. A learned value network replaced them, admitted because the policy it induces beats greedy. Qwen3.5-4B trained with SFT and then GRPO on its labels beats greedy on each of three training seeds: 69.7% [65.9, 73.2] over 600 games, every seed's lower bound above 50%, and 81.2% [77.8, 84.1] with two expert rules filtering the menu at play time. The value-network reference agent wins 87.5% [82.2, 91.4] against greedy, with its look-ahead's dice averaged as in the public oracle, and the trained model wins 7.0% against it (measured against the agent's dice-peeking variant, which is within a point of it against greedy). The main lesson: validate an oracle by the policy it induces, not by its fit statistics, and validate the engine by replaying what the agents actually did. A 27B run of the same recipe is in progress.

1. Introduction

Most evaluation work for language models assumes that a correct answer exists and can be checked. Many domains in which one would like to measure and train decision-making have no such checker: the rules are long, outcomes are stochastic, information is hidden and the consequence of a single decision is visible only hundreds of steps later. The work reported here asks what it takes to make such a domain evaluatable, so that a leaderboard number means something and a training signal can be trusted, and what goes wrong along the way.

Twilight Struggle is a demanding but tractable test case for four reasons.

1. **Hidden information.** Each player sees their own hand but not the opponent's or the deck order. Card counting is a real skill, so an agent has to reason under uncertainty that the observation makes explicit but does not remove.
2. **Stochasticity.** Coups, realignments and the space race are resolved by dice, and the deal is random. Two fixed agents need hundreds of games to separate.
3. **Long horizon of small decisions.** A game decomposes into a stream of atomic decisions (choose a card, choose its use, choose each target), about 360 in an average game and about 590 in one that lasts all ten turns (Section 3.1), each with a small legal set. The credit-assignment problem is severe: one coup at the wrong DEFCON level ends the game immediately, while most decisions shift the win probability by a few hundredths.
4. **A scripted baseline exists.** A position-scoring greedy agent plays a coherent game and beats a random agent that avoids immediate losses 81.5% of the time (79% as USSR, 84% as USA). It gives every result an anchor that does not depend on any model.

Contributions.

- An open environment (ts-env, MIT) with a closed 237-item action vocabulary, a deterministic text rendering for language models and a numeric encoding for learned policies, both derived from the same observation function.
- A single-step benchmark of 847 held-out decisions with a reproducible oracle reward, game-clustered confidence intervals, model-free baseline rows and a measured ceiling; and a full-game benchmark of 200 games against greedy with Wilson intervals and a decomposition of losses by cause.
- A documented sequence of three oracles: two rejected with the measurements that rejected them, and a learned value network whose admission criterion is the strength of the policy it induces.
- Training results for Qwen3.5-4B over three seeds (and a capacity control with Qwen3.8-27B) showing what supervised fine-tuning and GRPO each contribute, together with a list of negative results and engineering pitfalls (Section 8).

2. Summary of results

Table 1 collects the headline numbers. Full-game rows are 200 games against greedy per training seed, 100 per side, with 95% Wilson intervals, on ts-env commit 6791b34 unless marked; single-step rows are on the 847 held-out decisions scored by the public oracle (value network round 2 with dice averaged over 32 reseeded look-aheads, Section 4.3), with 95% intervals from a bootstrap over the 58 source games. The reference line is the value-greedy agent with rules whose look-ahead averages the dice over 4 reseeded samples, the dice treatment of the public oracle; the value-greedy row without rules and the 200 games of the last row used the variant whose look-ahead sees the real dice, which against greedy is within a point of the reference line (Section 4.3).

Table 1. Headline results.

System	Single-step oracle_value (n = 847)	Full games vs greedy
Random legal choice (expected)	0.495 [0.486, 0.503]	not applicable
Greedy baseline	0.590 [0.564, 0.616]	50% by symmetry
Qwen3.8-27B zero-shot	0.592 [0.565, 0.616]	0 of 4 in an early probe
Gemini 3.1 Pro preview, reasoning low (zero-shot)	0.610 [0.592, 0.627]	not run
Qwen3.5-4B, SFT only (3,292 labels)	0.717 [0.695, 0.739]	17.5% [12.9, 23.4] *
Qwen3.5-4B, SFT + GRPO (47,894 labels), three seeds	0.788 to 0.804	69.7% [65.9, 73.2] (600 games)
same, with rule-masked menu at play time	not applicable	81.2% [77.8, 84.1] (600 games)
Value-greedy agent (value network round 2), no LLM	not applicable	76.0% [69.6, 81.4]
Value-greedy agent + rules (reference line)	not applicable	87.5% [82.2, 91.4]
Qwen3.5-4B best adapter (seed 42) vs the reference line	not applicable	7.0% [4.2, 11.4]

* Measured on ts-env 515622c.

Three readings follow. The trained 4B model is the first language-model configuration whose lower bound against greedy clears 50%, and it does so on each of three training seeds (lower bounds 59.7, 65.9 and 63.3); with the rule mask both sides clear 50% (USSR 85.7%, USA 76.7%). The value-network agent that labels its training data is still stronger, and against that agent the trained model almost never wins: the distillation gap is large. What replicates across seeds is the win rate, not the way the model loses (Section 6.4).

3. The environment and the tasks

3.1 Terminology: game, turn, action round, atomic decision

The report uses four nested units of play (Table 2). The first three are the rulebook’s, the fourth is the engine’s. *Turn* always means a game turn, never a single move or one player’s go; one player’s go is an *action round*; a *step* or a *decision* is always one atomic decision.

Table 2. Units of play. Per-turn and per-action-round counts are from 200 greedy self-play games on ts-env 6791b34; game lengths are from the games named in the table.

Unit	What it consists of	Measured size
Game	up to 10 turns (Early War turns 1 to 3, Mid War 4 to 7, Late War 8 to 10), then final scoring; ends earlier on DEFCON 1, 20 VP, control of Europe, a scoring card held at turn end, or one of a few event wins	about 360 atomic decisions on average (359.7 over the 10,000 round-3 data games, ts-env 515622c; 348 over the best adapter’s 200 games against greedy, ts-env 6791b34); about 590 in a game that reaches turn 10; many games end early (mean final turn 4.5 in the round-3 data games, 6.2 in the best adapter’s games)
Turn	the deal; the headline phase (each side chooses a headline card, simultaneously under the rules; the engine asks one side after the other and does not show the first card to the second chooser; then both events resolve); 6 action rounds per side in the Early War and 7 in the Mid and Late War (8 with a space-race bonus or North Sea Oil), USSR and USA alternating; turn-end checks (held scoring cards, military operations, DEFCON improves by one), which ask no decision	about 55 atomic decisions per completed turn
Action round	one side plays one card: choose the card, choose its use (event, influence, coup, realignment, space race), then each target, amount and confirmation that use requires; an event may also put a decision to the opponent	about 3 atomic decisions by the acting side (mean 3.3, median 3, 90th percentile 6, at most 10)
Atomic decision	one question from the engine, with a menu of legal options from the 237-item vocabulary (Table 3), roughly one mouse click; one step of the environment and one item of the single-step task	1

The unit matters for everything below. The single-step task scores one atomic decision; the full-game task plays all of them; the one-turn oracle of Section 4.2 plays on to the end of the current turn; and the look-ahead of the value-greedy agent (Section 4.3) reaches the end of the mover’s current action round and no further.

3.2 Engine, decision stream and vocabulary

The engine implements all 110 cards of the deluxe edition. Board and card data are extracted from the game’s own data files rather than transcribed, and the test suite (278 tests) includes an audit of every card’s DEFCON effect against those files. Everything an agent sees is a pure function of the game state: a single function `observe(state, player)` hides the opponent’s hand and the deck order, and two views are computed from its output, a numeric encoding for learned policies and a text rendering for language models. Because both views come from the same observation, they cannot disagree on facts or leak different amounts of information.

An action round is itself a tree of small choices. Representing a whole action round as one step would make the action space combinatorial (four operations points spread over 84 countries), so the engine instead yields one atomic decision at a time, each roughly one mouse click. The action vocabulary is closed and ordered, so index i means the same thing in every game (Table 3).

Table 3. The 237-item action vocabulary.

Category	Example key	Count
Card	card:Duck and Cover	110
Card use	use:coup	6
Country	country:West Germany	84
Region	region:Middle East	9
Number	number:3	13
Event-specific option	option:1	12
Yes, no, pass	pass	3

The numeric view gives an 84×27 country matrix, a 153-dimensional global vector and 110-dimensional card vectors (hand, discard, removed, effects, unseen cards weighted by their probability of still being in the deck, and cards of the opponent revealed by events), 3,081 dimensions when flattened, plus a 237-dimensional legal-action mask. The text view precomputes what models tend to get wrong: control status per region, the score each region would yield if scored now, the influence needed to control each country, coup success odds and the regions where DEFCON forbids coups. Rendering is deterministic, so the same position always tokenises identically. Who moves next is irregular (the headline phase, events that put a decision to the opponent, extra action rounds), so the environment follows the usual convention for multi-agent games with irregular move order: each step reports who is to move and returns the reward from that player’s perspective. Two reward modes exist: sparse (terminal ± 1) and `vp_delta`, which adds victory-point changes and is not policy-invariant, because control of Europe, DEFCON 1 and a scoring card held at turn end all end the game without a corresponding VP change.

3.3 Baseline agents

Three scripted agents are provided: `random`, `safe_random` (random but refusing two immediately losing actions) and `greedy` (a one-decision position heuristic). Over 200 games (seeds 7000 to 7099, once per side) `greedy` beats `safe_random` 81.5% [75.5, 86.3], 79% as USSR and 84% as USA. The first player has a real advantage (8 to 16 points in symmetric pairings), so every evaluation is run on both sides.

3.4 The single-step task

Decision states are sampled from 300 games between the nine pairings of {`greedy`, `safe_random`, `epsilon-greedy` with `epsilon` 0.2}. A decision qualifies if it occurs at turn 2 or later, has 3 to 40 legal options and is of one of five types: `headline`, `play_card`, `card_use`, `place_influence` or `coup_target`. Up to three decisions per type are drawn per game, and the split is by game (seed divisible by 5 goes to held-out). This gives 3,292 training decisions and 847 held-out decisions from 58 games (Table 4). A larger training set used later covers all 12 sampled decision types (76,405 labelled decisions, filtered to a value spread of at least 0.03 and capped at 6,000 per type, leaving 47,894); a full game has 17 decision types. The source games were played on `ts-env` commit 515622c, and the stored action histories replay exactly only on that commit; the task itself never replays them, because each item stores its rendered view and option values.

Table 4. Held-out decisions by type (n = 847 from 58 games).

<code>card_use</code>	<code>coup_target</code>	<code>headline</code>	<code>place_influence</code>	<code>play_card</code>
174	158	170	171	174

The model sees exactly the prompt used in full games (system prompt plus rendered view) and answers with an `ACTION:` line, which the parser treats as authoritative. The reward is the oracle’s value of the chosen option normalised within the decision, `oracle_value` = $(v - v_{\min}) / (v_{\max} - v_{\min})$, with 0 for an illegal answer and 1 when all options tie. Because the normalisation is within the decision, random choice scores about 0.5, not 0. Reported alongside are `top1` (argmax hit, ties count as correct), `regret` and `first-try legality`.

3.5 The full-game task

The model under test plays 100 games as each side against greedy on seeds 5000 to 5099, with the same prompt and sampling as the single-step task (thinking off, temperature 0.6, 256 tokens). The primary metric is the win rate with a 95% Wilson interval; each game is an independent binomial trial. Every result is also broken down by side and by cause of loss (the model lowering DEFCON to 1 itself, a scoring card still in hand at turn end, VP and other endings), and wins are split into greedy’s own DEFCON suicides and wins on the board. At 200 games the Wilson half-width near 50% is about 7 points, so a lower bound above 50% needs about 115 wins (57.5%).

4. Three oracles, two rejected

The single-step task needs a value for every legal option of every decision. No ground truth exists, so the value has to come from an oracle, and the oracle’s quality bounds everything measured or trained with it. We built three. The first two failed in different ways, each detected by a different check; the third was admitted by a check that the first two could not even be subjected to.

4.1 Rollouts to game end: too noisy

The specification we started from valued an option by the mean outcome of k rollouts to game end with weak continuation policies (epsilon-greedy on both sides, common random numbers across options), $k = 64$ on held-out. To measure reliability, 120 held-out decisions were labelled twice with independent seeds. The noise ceiling is the score that the second labelling’s argmax obtains on the first labelling, which is the best any model could do.

Table 5. Test-retest reliability of the rollout oracle and the one-turn oracle.

Quantity	Rollout to game end, $k = 64$ ($n = 120$)	One-turn greedy, $k = 16$ ($n = 150$)
Within-decision Pearson r , mean	0.22	0.77
Within-decision Pearson r , median	0.20	0.97
r by type: card_use, coup_target, headline, place_influence, play_card	0.49, 0.27, 0.29, 0.02, 0.03	0.89, 0.89, 0.65, 0.67, 0.76
Argmax agreement between labellings	32%	68%
Noise ceiling	0.632	0.905
Random legal choice (expected)	0.522	0.513

A ceiling 0.11 above random leaves no room for a leaderboard. Both local models scored at the random line (0.507 for the 27B, 0.510 for the 4B), which says nothing about the models. Five oracle variants were then compared on 40 held-out decisions labelled twice (Table 6).

Table 6. Oracle variants (40 decisions, each labelled twice).

Variant	Rollout policy	Horizon	k	Mean r	Median r	Argmax agreement	Seconds per decision
full_eps64	epsilon-greedy	game end	64	0.24	0.28	22 of 40	364
full_greedy16	greedy	game end	16	0.26	0.20	21 of 40	67
turn1_eps32	epsilon-greedy	end of this turn	32	0.57	0.78	21 of 40	61
turn2_eps32	epsilon-greedy	end of next turn	32	0.46	0.52	13 of 40	89
turn1_greedy8	greedy	end of this turn	8	0.73	0.88	27 of 40	14

The signal is lost in the horizon, not in k . Played out by weak policies, one atomic decision becomes a coin flip; stopping at the end of the current turn and scoring the position keeps the decision’s consequence visible.

4.2 One-turn greedy rollouts: reliable but biased

The second oracle plays greedy on both sides until the turn ends and scores the mover’s position with greedy’s own position function. It is reliable (Table 5), cheap and discriminative: on it, greedy scores 0.567, Gemini 3.5 Flash 0.575, Gemini 3.1 Pro preview 0.566, Qwen3.8-27B 0.530, Qwen3.5-4B 0.500 and random 0.488. A 4B model trained with SFT and then GRPO on these labels reached 0.676 [0.651, 0.700], above every other row with non-overlapping intervals.

In full games against greedy the same model won 20.5% [15.5, 26.6]. Of its 159 losses, 78 were DEFCON suicides and 41 were scoring cards held to turn end, and the second class was created by training: the untrained model never lost that way, the SFT model lost 60 games that way. The one-turn oracle values a scoring card by its immediate VP, so in a losing region holding it looks best, and the turn-end penalty lies beyond the horizon of most sampled decisions. The model had become the best single-step player in the table and lost 80% of its games. Reliability is necessary, not sufficient: an oracle can measure the wrong thing consistently.

4.3 A learned value network

The third oracle is a learned value function $V(s)$, the probability that the observing side wins, trained on game outcomes. Conceptually it is the rollout oracle with the averaging done once, in advance, across millions of positions: the noise of individual rollouts becomes function-approximation error, which more data and better play can reduce.

Data. Round 1 played 100,000 games between a policy mixture (greedy 40%, epsilon-greedy 0.1 25%, epsilon-greedy 0.3 20%, safe_random 15%, drawn independently per seat), recording 4% of decision states from both perspectives: 2.42 million non-terminal positions, generated in 16 minutes on 56 workers. Labels are the final outcome (1, 0.5 or 0) for the observing side. Later rounds placed the value-greedy agent in half the seats (Table 7).

Model. A 512×2 MLP on the 3,081-dimensional encoding, dropout 0.3, weight decay $1e-3$, held-out split by game (seed divisible by 10). Three decisions mattered more than architecture. First, overfitting precedes capacity: positions from one game share a label and the card vectors fingerprint the game, so on 3,000 games a larger MLP was worse than a constant predictor, and even at 100,000 games the best epoch is the first. Second, terminal states are excluded from training, because 58% of games end by DEFCON and the encoding does not record who triggered it; at use time the exact outcome is read from the rules. Third, one-ply evaluation over atomic actions is not one-ply over moves: after use:coup at DEFCON 2 the network rated the state 0.69, although every remaining target was a battleground and the next click lost the game. The value-greedy agent therefore lets greedy complete the mover’s remaining consecutive clicks (up to 12) before reading V .

How far the look-ahead reaches. For each legal option, the value-greedy agent and the labeller alike clone the game, apply the option, and let the greedy heuristic make the mover’s remaining atomic decisions of the card in play, up to 12 of them (COMPLETE_STEPS), stopping at the first decision that belongs to the opponent; they then read V on that state, or take the exact result if the game has ended. In the units of Table 2 the depth is the rest of the mover’s current action round: the card being played is finished, but no decision of the opponent is simulated, neither its reply nor its next action round, and the turn is not played out. (At a headline, the second chooser’s look-ahead also resolves both headlines with the opponent’s real card, the headline peek listed among the open issues in Section 9.) The look-ahead is shallow, but it is not a lookup of an immediate score, for three reasons. First, hidden information: the opponent’s hand and the deck order are unknown, so a position has no known value, only a probability of winning that V has to learn from outcomes; a look-ahead that used the real hidden state would be cheating (the headline peek above is the one place where it still does). Second, chance: coups, realignments and space-race attempts inside the completed card are resolved by dice, so an option leads to a distribution of states, and the public oracle averages V over 32 reseeded clones. Third, chains of atomic decisions: the consequence of an option is usually decided several clicks later (use:coup at DEFCON 2 is safe or fatal depending on the target chosen next), so a one-click look-ahead would score a half-played card. The mapping from option to consequence therefore has to be completed and sampled before it can be scored. Looking further, into the opponent’s reply, multiplies the cost by the opponent’s options and by

the hidden-information samples: on the current engine a two-ply expectimax costs about 33 s per decision instead of 0.42 s (Section 9), too slow for labelling 76,405 decisions or for 200-game validity tests.

Table 7. Value network data and fit (held-out split by game).

Round	Games in training set	Non-terminal positions	Seats	Held-out AUC	Held-out log-loss
V1	100,000	2.42 M	policy mixture	0.759	0.583
V2	about 109,000	2.82 M	+ 10,000 games with V1 value-greedy in half the seats	0.761	0.583
V3	118,865	3.33 M	+ 10,000 games with V2 value-greedy + rules in half the seats	0.774	0.574

V1’s AUC by turn is 0.71, 0.79, 0.80 and 0.81 for turns 1 to 2, 3 to 4, 5 to 6 and 7 to 10; a logistic regression on the 153 global features reaches 0.741. Calibration is good in the middle and slightly overconfident at the extremes (V3: predicted 0.046 observed 0.108; predicted 0.953 observed 0.921).

Admission criterion. AUC on outcomes of weak mixed play is not the quantity that matters. The test is whether the agent that picks the option of highest V beats greedy, with no language model involved (Table 8). This test is unavailable to the rollout and one-turn oracles, which are too slow or too myopic to play with, and it is what makes V more than a fitted classifier.

Table 8. Validity test: value-greedy agent against greedy (200 games, 95% Wilson). Rows marked * were measured on ts-env 515622c. The reference line (marked †) averages the look-ahead’s dice over 4 reseeded samples; every other row lets the look-ahead see the real dice, an upper bound for a fair one-ply agent.

Agent	V1	V2	V3
Value-greedy	74.5% [68.0, 80.0] *	76.0% [69.6, 81.4]	89.5% [84.5, 93.0]
+ rules, first version of B2	96.5% [93.0, 98.3] *	90.5% [85.6, 93.8] *	not run
+ rules, narrowed B2 (current)	not run	87.5% [82.2, 91.4] †	93.0% [88.6, 95.8]

The same V2 agent with rules and the dice-peeking look-ahead (the default of `oracle/value_agent.py`) wins 88.0% [82.8, 91.8], within a point of the reference line (paired by seed 22 : 18, $p = 0.64$).

The fit statistics barely separate the rounds: V2 to V3 raised the reported AUC by 0.013, but on the same 327,370 held-out positions the two models score 0.770 and 0.774, with the gain confined to round-3 games. The induced policies separate clearly: without rules V3 beats greedy 89.5% against V2’s 76.0% (paired by seed, 41 games won only by V3 against 14 won only by V2, sign test $p = 0.0004$); with rules, both with the dice-peeking look-ahead, V3’s 93.0% is 5 points above V2 (23 : 13, $p = 0.13$).

The labels nevertheless stay on V2. Every model trained in this report was trained on V2 labels, the reference agent of the second reference line is V2 + rules, and in the configuration that matters for both roles (with rules) the two rounds are not separated at 200 games. A V3 oracle would be published as a new, versioned label set.

The public oracle. Labelling an option means cloning the game, applying the option, letting greedy finish the mover’s sub-decisions and reading V. Two random sources hide in a naive version of that procedure: a clone that replays the real seed sees the dice the real game will roll, and greedy tie-breaks drawn from one random stream shared across decisions make the labels depend on the

worker schedule. The public labels remove both: every option whose evaluation consumes the game’s random stream, or hits a completer tie, is evaluated on 32 clones reseeded with a hash of (public salt, game, step, option, sample) and V is averaged. A label is then a pure function of position, option and salt; two labelling runs with different scheduling agreed on all 847 decisions bit for bit. The ceiling of `oracle_value` is 1.0 by construction; the Monte Carlo noise of $k = 32$, measured by relabelling with two other salts, is small (their argmax scores 0.988 to 0.989 on the public labels).

None of this makes V honest. V estimates who wins among mixed-policy players, not under optimal play; it sees consequences through greedy’s completion of the mover’s action round; 85% of its training positions still come from round-1 mixture play; and its look-ahead still sees the opponent’s headline when it chooses second (Section 9, open issues).

5. Expert knowledge: features versus constraints

Strategy notes written by strong human players supply two kinds of knowledge: quantities a player should track (scoring cards in hand against action rounds left, over-control, military-operations shortfall, DEFCON-lowering cards in hand, and others) and hard rules. The two kinds behave in opposite ways.

As features, they add nothing. Adding 26 hand-crafted features to V’s input and retraining on the same 100,000 games changes held-out AUC from 0.7607 to 0.7604. The network already recovers these quantities from the 3,081 inputs.

As constraints, two rules add 3.5 to 22 points. The value-greedy agent’s own losses pointed at two causes (32 DEFCON losses and 17 held scoring cards in V1’s 51 losses), and both correspond to hard rules in the notes:

- **B1.** Play a scoring card when the action rounds left in the turn are no more than the scoring cards in hand.
- **B2.** At DEFCON 2 or lower, play the opponent’s DEFCON-lowering cards only into the space race, never headline such a card, and exclude battlegrounds from coup targets.

Applied as a mask on the options before V, they lift value-greedy from 74.5% to 96.5% (V1, first B2, ts-env 515622c), from 76.0% to 87.5% (V2, narrowed B2; the rules row is the non-peeking reference line, the peeking one is within a point) and from 89.5% to 93.0% (V3, narrowed B2). V is not ignorant of these losses, it is not sharp about them: a game ending with a held scoring card occurs in about 0.1% of round-1 games, so V never learns that the outcome is certain. A rule turns that rare event into a prohibition. The interface for this kind of knowledge is the action space, not the input.

Rule width must match the game’s rule. The first version of B2 forbade every coup at DEFCON 2. Value-greedy tolerated it (96.5%), because V still ranked the remaining options. When the same rule was folded into the single-step labels (forbidden options set to the decision minimum) and a 4B model was trained on them, its single-step score was unchanged (0.731 against 0.737 on V1 labels) but its full-game win rate fell from 51.0% to 23.5% [18.2, 29.8], with 125 games lost on VP: the model had generalised the floor into “never coup”, conceding military operations every turn. The narrowed B2 above is the rule in use; it changes 326 of the 847 held-out labels instead of 410. A rule placed in a reward has to be exactly as narrow as the rule in the game, because a learner turns any excess width into a habit.

6. Training language models against the oracle

6.1 Recipe

All adapters are LoRA (rank 32, alpha 64, all linear projections) on Qwen3.5-4B unless stated. SFT trains on ACTION: `<oracle argmax>` with loss on the completion only. GRPO (TRL 1.14) starts from the SFT adapter: 200 steps, learning rate 1e-5, eight generations per prompt at temperature 1.0, reward `oracle_value`, DAPO loss aggregation, group-normalised advantages, no KL term, colocated vLLM generation. With one optimisation pass per batch the importance ratio stays at

1 and clipping is inactive, so the update is in effect REINFORCE with a group-mean baseline and group-standard-deviation scaling. Checkpoints are never selected on the held-out set; the final step is always reported. All training labels are V2 labels made with the earlier oracle definition (look-ahead with the real dice, completer ties unseeded; Section 9); the models are scored on the public labels.

6.2 SFT teaches targets, GRPO teaches conditions

Table 9 shows that the dominant effect is not the label source or the data size alone but whether GRPO follows SFT.

Table 9. Full games against greedy (200 games, 100 per side, 95% Wilson). Loss columns give the three main causes; final scoring, wargames and similar endings are omitted, so they need not sum to the losses. The two big-data rows (seed 42) were measured on ts-env 6791b34, the other rows on ts-env 515622c.

Adapter	Labels	Train decisions	Win rate	USSR	USA	Losses: own DEFCON	Losses: held card	Losses: VP
Untrained 4B	none	0	7.0% [4.2, 11.4]	5%	9%	131	0	53
SFT	one-turn greedy	3,292	12.5% [8.6, 17.8]	8%	17%	86	60	24
SFT + GRPO	one-turn greedy	3,292	20.5% [15.5, 26.6]	18%	23%	78	41	36
SFT + GRPO	V1	3,292	51.0% [44.1, 57.8]	62%	40%	26	13	50
SFT + GRPO	V1, first-B2 floors	3,292	23.5% [18.2, 29.8]	n.r.	n.r.	8	20	125
SFT only	V2	3,292	17.5% [12.9, 23.4]	n.r.	n.r.	155	2	5
SFT + GRPO	V2	3,292	42.0% [35.4, 48.9]	47%	37%	46	0	56
SFT only	V2	47,894	21.0% [15.9, 27.2]	n.r.	n.r.	140	n.r.	n.r.
SFT + GRPO, seed 42	V2	47,894	66.5% [59.7, 72.7]	79%	54%	8	28	15
same, rule-masked menu	V2	47,894	80.5% [74.5, 85.4]	86%	75%	5	1	20
27B SFT only	V2	3,292	15.0% [10.7, 20.6]	14%	16%	165	2	3

n.r.: not reported in the source records.

SFT-only adapters lose 140 to 165 games by lowering DEFCON to 1 themselves; the SFT + GRPO adapters on V labels lose 8 to 46 that way. Replays show the mechanism: at DEFCON 2 the SFT model plays use:coup and then a battleground (Nigeria, Panama), the same target it learned to prefer at DEFCON 3 and above. SFT on the argmax gives one positive example per decision, so the DEFCON condition has to be inferred from two unrelated sets of positives. GRPO samples the suicide, receives the exact terminal value 0 from V while the sibling samples receive 0.5 to 1, and the group-normalised advantage pushes down that key in that prompt. Negative examples carry the conditional knowledge, and SFT has none. How much of the loss class GRPO removes is seed-dependent, though (Section 6.4). The single-step score barely registers this (V1 labels: SFT 0.704 against SFT + GRPO 0.737); the full-game win rate triples.

An earlier comparison made the opposite point by accident: SFT on 47,894 decisions (21.0%) against SFT + GRPO on 3,292 decisions (51.0%) suggested that more data hurt. The two runs differed in two variables. The control, SFT only on 3,292 decisions (17.5%), showed that the difference was GRPO.

6.3 Data matters once GRPO is in place

With GRPO following SFT, the 47,894-decision set raises the full-game win rate from 42.0% [35.4, 48.9] to 69.7% [65.9, 73.2] pooled over three seeds (seed 42: 66.5%; intervals disjoint) and the single-step score from 0.741 to 0.800 (paired +0.059 [+0.040, +0.077]). The gain is on the board, not from greedy’s accidents: greedy’s own DEFCON suicides account for 59, 48 and 46 of the wins of the three V-labelled SFT + GRPO adapters of Table 9, while wins on the board go from 43 and 36 to 87. It is concentrated in card choice (headline 0.884 against 0.811, play_card 0.760 against 0.649), the decision types the small set covered thinnest. The remaining weakness moves: 28 of the seed-42 adapter’s 67 losses are scoring cards held at turn end, 19 of them as USA.

That is the loss class B1 targets. With B1 and B2 applied only as a menu filter at play time, never in training, the same adapter wins 80.5% [74.5, 85.4], USSR 86% [77.9, 91.5] and USA 75% [65.7, 82.5], both side lower bounds above 50%. Held-card losses fall from 28 to 1. The same mask on the V1 adapter moved 51.0% to 53.0% [46.1, 59.8] (ts-env 515622c). Two quantities explain the difference: the share of losses the mask addresses (13 of 97 on V1, 28 of 67 here) and whether the underlying policy converts the rescued games (V1’s rescued games were lost on VP a few turns later; here the rule-class losses fall by 30 and the wins rise by 28).

6.4 Seeds

Table 10. Three seeds of the big recipe (SFT on 47,894 V2 labels, then GRPO 200 steps). Single-step on 847 held-out decisions (public labels) with game-clustered 95% intervals; full games 200 against greedy per seed with 95% Wilson, ts-env 6791b34.

Seed	SFT-only start	SFT + GRPO	GRPO gain (paired)	Full games	as USA	Own- DEFCON / held-card losses	Full games, rule-masked
42	0.755 [0.734, 0.775]	0.800 [0.782, 0.818]	+0.045 [+0.027, +0.062]	66.5% [59.7, 72.7]	54%	8 / 28	80.5% [74.5, 85.4]
2	0.741 [0.720, 0.762]	0.804 [0.784, 0.823]	+0.063 [+0.041, +0.084]	72.5% [65.9, 78.2]	59%	20 / 22	86.5% [81.1, 90.6]
3	0.739 [0.716, 0.761]	0.788 [0.766, 0.809]	+0.049 [+0.025, +0.073]	70.0% [63.3, 75.9]	61%	23 / 8	76.5% [70.2, 81.8]
pooled, 600 games				69.7% [65.9, 73.2]	58.0% [52.3, 63.4]	51 / 58	81.2% [77.8, 84.1]

The claim replicates: every seed’s Wilson lower bound clears 50%, with and without the mask, and GRPO’s single-step gain is positive on every seed. Unmasked, the three win rates are consistent with one underlying rate; masked there is a weaker seed effect (chi-square 6.6, $p = 0.036$), with seed 3 converting fewer rescued games. As USA alone the unmasked policy is borderline (58.0% pooled, 54% for seed 42); the masked policy clears 50% on both sides (USA 76.7%).

The way the model loses does not replicate. Seed 42 loses 8 games by lowering DEFCON to 1 itself, seeds 2 and 3 lose 20 and 23; held-card losses run the other way (28, 22, 8). Seed 42 alone would suggest that GRPO removes DEFCON suicide; the other seeds show that GRPO cuts this loss class from the SFT-only level of 140 to 165 by a large but seed-dependent amount, and that the reproducible quantity is the win rate. The single-step score does not predict the full-game ordering of the seeds.

6.5 Data or capacity

To test whether model size limits what SFT extracts from the labels, Qwen3.8-27B was trained with the identical LoRA SFT recipe on the same 3,292 V2 labels as the 4B sft_v2 adapter (Table 11).

Table 11. Capacity control: identical SFT recipe on 3,292 V2 labels (single-step on the public labels; full games measured on ts-env 515622c).

Model	Final eval loss	Single-step oracle_value (n = 847)	top1	Full games vs greedy (n = 200)	Own-DEFCON losses
Qwen3.8-27B zero-shot	not applicable	0.592 [0.565, 0.616]	0.224	not run	not applicable
Qwen3.8-27B LoRA SFT	0.2161	0.734 [0.710, 0.756]	0.416	15.0% [10.7, 20.6]	165 of 170
Qwen3.5-4B LoRA SFT	0.2173	0.717 [0.695, 0.739]	0.375	17.5% [12.9, 23.4]	155 of 165
Paired difference, 27B minus 4B (bootstrap over 58 games)		+0.016 [-0.008, +0.040]	+0.040 [+0.007, +0.073]		

The two models pick an option of equal oracle value on only 48% of decisions: they are different policies of nearly equal average quality, the 27B slightly more often on the argmax. At this data size, and for imitating the V argmax, the extra parameters buy little on the single-step score and nothing in full games; the larger model also does not learn the DEFCON condition from SFT alone. The gains in this project came from labels (one-turn to V1: 20.5% to 51.0%), from GRPO, and from data with GRPO.

6.6 The distillation gap

The value-greedy agent with rules, the source of the labels plus the same masks, is the second reference line (87.5% against greedy). Against it, the best adapter (seed 42, unmasked) wins 14 of 200 games, 7.0% [4.2, 11.4] (USSR 3, USA 11), and 13 of the 14 wins are the opponent’s own DEFCON drops. These games were played against the agent’s dice-peeking variant (same network, rules and pruning; the opponent of --opponent value_rules), whose rate against greedy is within a point of the reference line (Section 4.3); the non-peeking agent has not been played against the adapter. Of its 186 losses, 116 are on VP, 39 on a held scoring card and 28 on its own DEFCON drop. The student imitates single decisions of a teacher it cannot approach as a player; the gap is mostly positional play. An earlier V1 adapter won 5.9% [3.4, 10.3] (ts-env 515622c).

7. Evaluation methodology

Each number in this report carries four companions: an interval computed over the right sampling unit, model-free baseline rows, a ceiling, and a decomposition. The rules below are the ones that changed a conclusion at least once.

- **Cluster the bootstrap by game.** The 847 held-out decisions come from 58 games, and decisions within a game share a deck, a board trajectory and an opponent. Intervals resample games (2,000 resamples), which gives roughly ± 0.025 against ± 0.015 for a naive per-decision interval. Paired comparisons between two models resample the same games for both.
- **Use Wilson intervals for win rates.** Games are independent binomial trials. The Wilson interval stays inside [0, 1] near the edges and fixes the decision rule: with 200 games, “beats greedy” requires about 115 wins.
- **Print the baseline rows.** Random legal choice (computed in expectation from the labels) and greedy placed on the same decisions are part of every table. Because oracle_value normalises within the decision, random sits near 0.5, and a model at 0.55 is barely above guessing.
- **Print the ceiling.** For a stochastic oracle, the ceiling is the retest score (0.632 for rollouts, 0.905 for the one-turn oracle). For V’s public labels it is 1.0 by construction, and the oracle’s own Monte Carlo noise is printed next to it (another salt scores 0.988). Without the ceiling a reader cannot tell a weak model from a noisy reward.

- **Seed every random source of the oracle and check bit reproducibility.** Labels made with unseeded completer tie-breaks reproduce only 82.8% of their argmaxes when the same command is rerun.
- **Decompose by type, side and cause.** Every diagnosis in this project came from a decomposition, not from a win rate: 41 held-card losses exposed the one-turn oracle’s horizon, 155 DEFCON losses exposed what SFT does not learn, 125 VP losses exposed an over-wide rule. Wins are also decomposed, because greedy’s own DEFCON suicides contribute a large and roughly constant share.
- **Split held-out by game and never select checkpoints on it.** Both the single-step split and V’s split are by game seed. GRPO is evaluated every 25 steps on 300 held-out decisions, but the last checkpoint is reported: on the V1 run, step 100 scored 0.742 and step 200 scored 0.711, a difference inside the noise of $n = 300$.
- **Treat single runs of a recipe as one sample.** The same small recipe on V1 and V2 labels gave 51.0% and 42.0%, intervals overlapping; both are read as parity with greedy, not as a regression. The three seeds of the big recipe show that even the loss decomposition of one run is a sample.
- **Validate the oracle independently of the model it trains.** The value-greedy validity test (Table 8) involves no language model.

8. Negative results and pitfalls

This section lists what did not work and what cost time, one or two sentences each. Most of these findings would not appear in a paper reporting only the final configuration, and several of them changed the direction of the project.

8.1 Methodological

1. **Few ties is not discrimination.** The rollout oracle almost never tied two options, which looked like resolving power; continuous noisy estimates never tie. The retest correlation (0.22) was the real measure.
2. **A reliable oracle can be wrong.** The one-turn oracle ($r = 0.77$) produced the best single-step model and a 20.5% full-game win rate, and training created a loss class (held scoring cards) the untrained model never had.
3. **AUC is not the admission criterion.** V2 to V3 changed AUC on the same positions by 0.004 and value-greedy by 13.5 points.
4. **Expert features are redundant, expert rules are not.** 26 features: AUC 0.7607 to 0.7604. Two rules: +3.5 to +22 points.
5. **An over-wide rule in the reward teaches a habit.** First B2 in the labels: 51.0% to 23.5%, 125 VP losses.
6. **SFT on the argmax does not learn conditions.** SFT-only adapters lose 140 to 165 games to DEFCON suicide regardless of data size or model size; GRPO cuts this to 8 to 46, by an amount that varies across seeds of one recipe (8, 20, 23).
7. **Changing two variables at once produced a false conclusion.** “More SFT data is worse” was “no GRPO”.
8. **A newer oracle round did not transfer at small data.** The 4B on V2 labels (42.0%) was not better than on V1 labels (51.0%) until the training set grew.
9. **Capacity did not help.** 27B SFT is level with 4B SFT on single-step (paired +0.016 [−0.008, +0.040]) and in full games (15.0% against 17.5%).
10. **Thinking mode did not fit the budget.** At 12,000 tokens, 64% of Qwen3.8-27B replies and 57% of Qwen3.5-4B replies were truncated inside the reasoning block; the finished 27B replies matched greedy (0.567) on a self-selected subset.
11. **GRPO from the base model dips first.** It spent its first 100 steps below zero-shot and ended level with SFT (0.593 against 0.597); GRPO from the SFT adapter did not dip.
12. **A mask’s value depends on the policy under it.** The same B1 and B2 mask gave +2.0 points on the V1 adapter (ts-env 515622c) and +11.5 on the big adapter (three seeds pooled, 69.7% to 81.2%).

13. **“Deterministic” oracles are not.** A clone that replays the real seed sees the dice; an unseeded tie-break makes labels depend on the worker schedule. Neither shows up in any fit statistic.

8.2 Harness and engineering

1. **Answer parsing.** Splitting replies on whitespace truncated keys containing spaces (country:West Germany) and silently rejected 27% of valid replies; not accepting menu numbers (ACTION: [3]) marked 82% of first replies of the 27B illegal, and accepting them raised legality to 99%.
2. **A rules bug found by a model’s rationale.** One event card had its DEFCON effect inverted, which turned a safe card into a losing one and reversed a published baseline conclusion. No test caught it; a language-model agent whose stated reason contradicted the engine’s verdict did. An audit of all 110 cards’ DEFCON effects is now a test.
3. **Fork and torch deadlock worker pools.** Every pool that touches torch uses the spawn start method.
4. **Game.clone() is O(history).** Clones replay the action history because running games hold generator frames. A late 40-option decision costs seconds, a round of 10,000 value-greedy games took 9.8 hours, and play prunes to greedy’s top 10 options (labelling never prunes).
5. **A wall-clock cap is load-sensitive.** With a 600-second per-game cap on a loaded machine, 35 to 37 of 200 validity games were aborted and the V3 rows read 12.5 to 13.5 points lower than with a 3,600-second cap. Aborts count as losses, so a time cap silently biases a win rate downward.
6. **TRL loads base weights in fp32 by default** when the trainer is given a model id; harmless for 4B, fatal for 27B, which needs an explicit bfloat16 dtype.
7. **The vision tower takes the KV cache.** Serving Qwen3.8-27B with vLLM defaults at 80% memory utilisation left a negative KV budget (−4.4 GiB) because of vision-encoder profiling and CUDA graphs for 256 sequences; serving the language model only with 32 sequences gave 9.4 GiB.
8. **A rules bug, unbounded realignment rolls, surfaced only when a replay review compared games with the written rules.** Greedy never realigns, so tests and thousands of greedy games passed; its only symptom, about 1% of value-greedy games looping inside an event, had been capped and counted as losses. Every number in this report not marked 515622c was produced after the fix.

9. Limitations and future work

Open issues. The engine asks the two headlines one after the other and does not show the first card to the second chooser, so language models cannot see it; but the look-ahead of the value-greedy agent and of the labeller clones the complete game state, so as second headline chooser it resolves both headlines with the opponent’s real card. This headline peek is not handled: it affects 90 of the 847 held-out labels (public labels included) and the headlines of the reference agent’s games. The models’ training labels use the earlier oracle definition (look-ahead with the real dice, completer ties unseeded); the models are scored on the public labels and in full games. Rebuilding dataset positions from their action histories needs ts-env 515622c, the version the data was generated on; full games run on 6791b34.

The next phase, in order of priority:

1. **Regenerate the data on the current engine and retrain.** The single-step positions, the training sets and V’s training games were all generated on ts-env 515622c, and the training labels use the earlier oracle definition; this is why two ts-env commits are pinned today. The next round (round 4) generates candidate decisions and positions on 6791b34, with V3 + rules and greedy in the seats, relabels the training and held-out sets with the public oracle definition, and retrains V and the language models (three seeds). Dataset, oracle and evaluation then pin one engine commit, and the legacy checkout 515622c is retired. The regenerated set should also cover more of the 17 decision types and the critical situations that baseline play makes

rare (DEFCON 2 with a coup option, scoring cards late in a turn); it will be published as a new version of the benchmark.

2. **Fix the headline peek.** At a headline where the opponent has already chosen, the look-ahead will draw the opponent’s card from the mover’s unseen headline-capable cards in each sample and average V, instead of resolving the real one. The same determinisation (resample everything the mover cannot see: opponent hand, deck order, dice) subsumes the dice treatment. The headline items are then relabelled and the reference line rerun, which bounds its effect on the 87.5%.
3. **Train V on stronger play.** V is trained mostly on weak play: 85% of its positions come from round-1 mixture games, 59% of which end on DEFCON, so it has seen few games decided by positional play and none by a strong player, although it is still clearly stronger than any model trained on it. Round 4 puts V3 + rules in the seats; later rounds should add the strongest agents available (trained adapters, search agents). Whether V3 is better than V2 with rules needs about 500 games per side.
4. **27B with the full chain.** A 27B model with SFT on the 47,894-decision set followed by GRPO is the clean second test of capacity (Section 6.5 tested only SFT on 3,292 decisions). This run is in progress; its results will be added to this report.

Further limitations:

- **No human anchor, and an Elo pool that is not transitive for LLMs.** Every strength number is relative to programmatic agents. A Bradley-Terry ladder over the scripted and value-net agents (measured on 515622c, peeking agents; refit pending) places V2 + rules at +357 [+318, +404] Elo over greedy and fits one scale well (deviance 4.8 on 4 degrees of freedom), but nothing in the pool is a person, so “+357” says nothing about strength against people. The language models do not fit that scale at all: the best adapter rates +149 from its games against greedy but about −188 from its games against V2 + rules, and adding the LLMs raises the deviance to 39.8 on 6 degrees of freedom. From the rates on 6791b34 the same two conversions give +119 and about −103 (the second through the rate of the peeking opponent the games were played against), still about 220 Elo apart. LLM strength is therefore reported per named opponent. The game’s built-in AI ships as a native library inside a commercial client and cannot serve as a programmatic opponent; a manual calibration match through a relay mode, or human games against the value agent, would provide the anchor.
- **Greedy is a weak opponent with a known flaw.** 189 of the 418 pooled wins of the best recipe (45%) are greedy lowering DEFCON to 1 itself, and as USA the unmasked policy is only borderline (58.0% [52.3, 63.4]).
- **Rules act at inference, not in training.** B1 and B2 filter the menu at play time. Putting them into training failed once (Section 5); doing it correctly, for instance as a narrow negative reward in GRPO, is untested.
- **Search is the obvious next route, and on this engine it is expensive.** The one-ply look-ahead costs about 0.42 s per decision and a full value-greedy game about 2.5 minutes, 93% of it in `Game.clone()`, which replays the history (92 ms per clone at step 600). Measured from that per-leaf cost: a two-ply expectimax with 8 hidden-information samples costs 33 s per decision, 3.4 hours per game, 21 hours for one 200-game validity test and 14 hours to relabel the 76,405-decision pool (10 minutes today); information-set MCTS with 200 simulations costs 8.4 s per decision and about 50 minutes per game; one AlphaZero-style round of 100,000 self-play games at 200 simulations is about 84,000 core-hours, 27 days on a 128-core machine, before any network training. The order of work is therefore an $O(1)$ state snapshot first (estimated to bring a game from 2.5 minutes to about 25 s), then a policy head for pruning and completion, then search; and the bottleneck search would amplify is V’s accuracy, not its depth.

10. Reproducibility

Code and data are split across two public repositories.

- **ts-env** (github.com/bingyang1132/ts-env, MIT): the engine, observation, encoding and rendering, the three baseline agents (`twilight.baselines`), the language-model harness, the terminal player and the replay viewer. Card and board data are extracted from a locally owned copy of

the game and remain the publisher's copyright.

- **ts-eval** (github.com/bingyang1132/ts-eval): everything built on top of it.

Path	Contents
data/	the 847 held-out decisions with the public labels (the leaderboard) and with the legacy labels; 3,292 training decisions (legacy labels); the 47,894-decision training set as a release download; checksums and label definitions
env/ts_single_step/	the single-step environment for verifiers (vf-eval against any OpenAI-compatible endpoint; no engine needed at evaluation time)
oracle/	V2 checkpoint, position generation, value-network training, the value-greedy agent and validity test, relabelling with the public oracle
eval/	single-step rows and summary with game-clustered intervals, the full-game runner (--rule-mask, --opponent value_rules)
leaderboard/	the tables
training/	SFT and GRPO scripts (TRL), LoRA configuration
third_party/	ts-env as two git submodules: the games commit 6791b34 and the data commit 515622c
report/	this report

The environment is a separate repository; ts-eval references two of its commits as submodules: third_party/ts-env at commit 6791b34, on which full games and the reference agents run, and third_party/ts-env-legacy at commit 515622c, the version the data was generated on, which scripts that rebuild dataset positions from their action histories need. Each script selects its submodule and prints the engine in use. Typical commands, from the ts-eval root:

```
git clone --recursive https://github.com/bingyang1132/ts-eval && cd ts-eval
pip install -r requirements.txt "verifiers>=0.3.1" datasets torch
```

```
# single-step leaderboard row for a served model, and the model-free rows
bash eval/run_zero_shot.sh <row> <model> <base_url> off 256
python eval/baseline_rows.py --data data/oracle_heldout_value_r2_public.jsonl
python eval/summarize.py
```

```
# public labels from the stored decisions (about 5 minutes on 32 CPU workers)
python oracle/relabel_with_value.py --value oracle/value.pt \
  --candidates data/oracle_heldout_value_r2_public.jsonl --out labels.jsonl
```

```
# validity test of V, the reference line (without the dice flags: the peeking variant)
python oracle/value_agent.py --value oracle/value.pt --games 100 --opponent greedy --
prune 10 \
  --rules --dice expect --k 4 --salt a --max-seconds 3600
```

```
# training
python training/train_sft.py --train data/oracle_train_value_r2_big.jsonl
python training/train_grpo.py --init-adapter <sft adapter>/final \
  --train data/oracle_train_value_r2_big.jsonl
```

```
# full games
python eval/play_vs_greedy.py --model <served name> --games 100 [--rule-mask | --
opponent value_rules]
```

The V2 checkpoint is in the repository; the LoRA adapters are not distributed. Hardware used: A100 accelerators for training and serving; game generation and full-game evaluation run on CPU workers (12 to 96).